阿里云

基于 AgentScope 2.0 Harness 构建企业级、分布式智能体

刘军

阿里云

CONTENT
目录

01 AgentScope 2.0 介绍

02 AgentScope Harness 核心设计

03 AgentScope Service 企业级智能体控制面

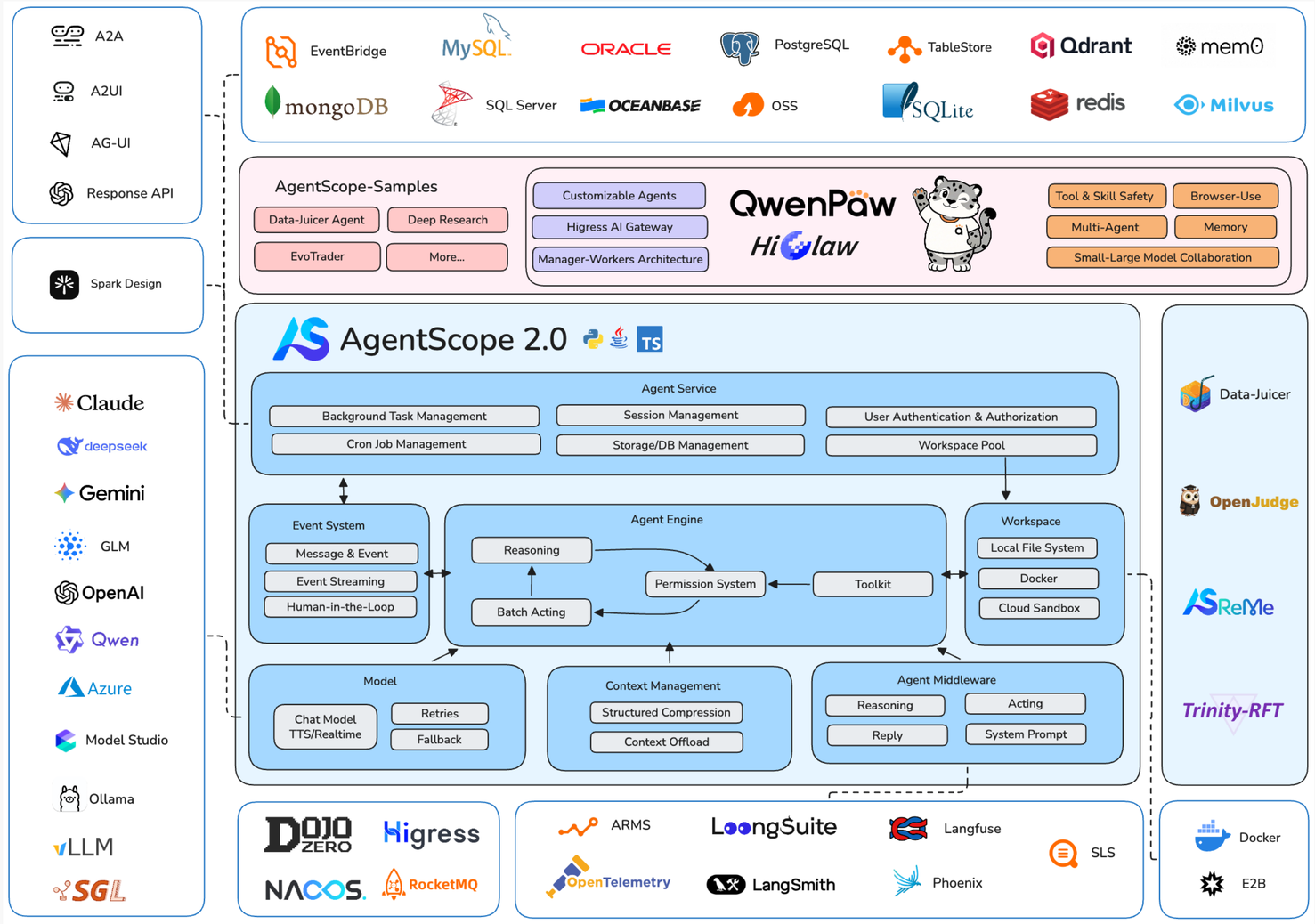
阿里云

01 AgentScope 2.0 介绍

AgentScope 概览

AgentScope 是阿里巴巴推出的一款面向企业级、分布式场景的智能体开发框架，覆盖 Python、Java、Typescript、Golang（建设中）语言实现。

AgnetScope 生态核心目标：提供一套覆盖“开发、部署、调优”全生命周期的生产级解决方案，让智能体应用的开发更简单、运行更稳定、效果更卓越。



阿里云

ReactAgent 核心组件

Agent 智能体

- ReAct 推理-行动循环引擎
- 多用户 / 多会话并发安全
- 流式事件 & 结构化输出
- 中断 / 恢复 & 人机交互

Model 模型层

- Credential + ChatModel 两层架构
- 5大厂商: DashScope / OpenAI / Anthropic / Gemini / Ollama
- Streaming & Thinking & 多模态
- 可扩展自定义 Provider

Tool 工具系统

- @Tool 注解 / ToolBase 继承
- MCP 协议集成 (STDIO / SSE / HTTP)
- Skill 热加载 Markdown 指令集
- Tool Group 按需激活/自管理

Context 上下文

- 无状态引擎 + AgentState 持久化
- RuntimeContext per-call 元数据
- Redis / MySQL 分布式状态共享
- 跨节点故障转移 & 会话恢复

Middleware 中间件

- 5 个生命周期 Hook 位置
- 洋葱式(Onion) + 变换式(Transformer)
- OpenTelemetry 全链路追踪
- 限速 / 回退 / 动态 Prompt

Permission 权限

- Rules + Mode + Built-in Checks
- 5 种模式: DEFAULT / EXPLORE / BYPASS / ACCEPT_EDITS / DONT_ASK
- 建议规则自动生成 & 持久化
- 危险路径不可绕过保护

Message & Event

- Msg 类型化内容块体系
- AgentEvent 流式增量传输
- Start → Delta → End 生命周期
- SSE 推送 & 断点重建消息

阿里云

1.0 -> 2.0 迁移指南

✓ 总体兼容 — 平滑升级

核心 API 保持一致
ReActAgent Builder 模式不变，model / toolkit / sysPrompt 等参数平滑迁移

@Tool 注解完全兼容
已有 @Tool / @ToolParam 标注的工具类无需任何修改即可在 2.0 中注册使用

MCP 客户端无缝衔接
McpClientBuilder API 不变；STDIO / SSE / Streamable HTTP 三种连接方式保持兼容

Model 层 Provider 兼容
DashScope / OpenAI / Anthropic / Gemini / Ollama 各 ChatModel builder 接口不变

消息体系向后兼容
UserMessage / AssistantMessage / SystemMessage 构造方式不变

✗ 破坏性变更 — 必须改造

状态管理架构重构
无状态 Agent 引擎：AgentState 按 (userId, sessionId) 寻址，不再绑定单一 agent 实例

RuntimeContext 替代旧调用模式
call() 必须传入 RuntimeContext；per-call 元数据不再挂在 agent 实例上

AgentStateStore 强制配置
分布式部署(sandbox)下必须配置 Redis / MySQL 等分布式状态后端，否则 build() 抛异常

Middleware 注册方式变更
新增 5 层 hook 体系；旧回调/拦截器需迁移为 MiddlewareBase 实现

Permission System 全新引入
工具执行前置权限检查为必选项；需配置 PermissionContextState（至少选择 Mode）

⚠ 已废弃 — v2.1 将移除

Memory 接口
InMemoryMemory / LongTermMemory 等标记 @Deprecated(forRemoval=true)

→ 迁移到 AgentState.getContext() + AgentStateStore

ToolExecutionContext
标记 @Deprecated；底层自动桥接到 RuntimeContext，老代码暂不失效

→ 迁移到 RuntimeContext

ToolCallParam.getContext()
已废弃

→ 迁移到 getRuntimeContext()

ImageBlock / AudioBlock / VideoBlock
仍兼容但新代码建议统一使用 DataBlock

→ 迁移到 DataBlock + mediaType

旧 interrupt() 无参重载
单 session 场景仍有效，多 session 下行为不确定
→ 迁移到 interrupt(userId, sessionId)

阿里云

02 AgentScope Harness 核心设计

阿里云

AgentScope Harness 总体架构

在 ReActAgent 之上，把长期运行 agent 必备的工程能力打包

应用 · 用户请求

HarnessAgent

薄包装 · 能力叠加在 ReAct 循环的关键时机 · 内部 middleware 顺序固定

Workspace
人格 / 知识

Memory
双层长期记忆

Compaction
上下文压缩

Subagents
子 agent 编排

Filesystem
本机/KV/沙箱

Skills
技能装配

Plan Mode
只读思考 +HITL

Channel / Gateway
会话 · 路由 · SSE

Permission
工具白名单

ReActAgent · 推理循环（core）· Harness 不改写此层，只叠加钩子

共享对象（能力间零耦合，只通过这三者通信）

RuntimeContext
userId · sessionId · extra

Workspace
AGENTS.md · MEMORY · skills

AgentStateStore
跨请求恢复运行时状态

AgentScope Harness 快速体验

```xml
<dependency>
    <groupId>io.agentscope</groupId>
    <artifactId>agentscope-harness</artifactId>
    <version>${agentscope.version}</version>
</dependency>
```

```java
public class FirstAgent {
    public static void main(String[] args) {
        HarnessAgent agent = HarnessAgent.builder()
                .name("note-taker")
                .sysPrompt("你是一个帮助用户做笔记的助手。")
                // 字符串形式由 ModelRegistry 解析 —— 自动读取 DASHSCOPE_API_KEY;
                // 切换其他厂商时改用 "openai:gpt-5.5"、"anthropic:claude-sonnet-4-5"、
                // "gemini:gemini-2.0-flash" 或 "ollama:llama3"。
                .model("dashscope:qwen-plus")
                .workspace(Paths.get(".agentscope/workspace"))
                .compaction(CompactionConfig.builder()
                        .triggerMessages(30)
                        .keepMessages(10)
                        .build())
                .build();

        RuntimeContext ctx = RuntimeContext.builder()
                .sessionId("demo-session")
                .userId("alice")
                .build();

        // 第一轮：自我介绍 + 当天的事
        agent.call(new UserMessage("我叫天宇，今天准备一个关于 ReAct 的技术分享。"), ctx).block();

        // 第二轮：同 sessionId，自动恢复上一轮状态后回答
        agent.call(new UserMessage("我叫什么？我今天要干什么？"), ctx).block();
    }
}
```

Workspace -- 智能体进化的 Source of Truth

「智能体是什么」+「学到了什么」都是文件

内容按生命周期分三类

静态资产

工程师编辑

`AGENTS.md · knowledge/ · skills/ · subagents/ · tools.json`

运行时文件

框架 / agent 写

`agents/<id>/sessions/ · agents/<id>/tasks/ · plans/`

长期记忆

agent + 后台任务

`MEMORY.md · memory/YYYY-MM-DD.md`

四条设计线索

① **定义与进化都是文件**

不散落代码、不绑数据库；一个目录拷走 = 完整 agent

② **生命周期三分**

静态 / 运行时 / 长期记忆 · 走不同读写路径

③ **原生多租户隔离**

IsolationScope: SESSION / USER / AGENT / GLOBAL

④ **Workspace ⊥ Filesystem**

同一份目录布局 → 三种物理后端可切换

抽象文件系统 – Workspace 的物理载体

同一份逻辑目录，三种物理后端，agent 代码零改动

Workspace · 逻辑目录布局

AbstractFilesystem 接口 · WorkspaceManager 路由

Local +Shell

默认

✓提供 Shell

- Overlay: workspace +project
- 路径策略 ROOTED / SANDBOXED
- 单进程 · 本机开发 · 信任环境
- 快、简单、无外部依赖

Remote KV

多副本

✗ 不提供 Shell

- Redis · JDBC · OSS · Nacos
- 本机模板 + 远端覆盖（两层读）
- 多副本共享 MEMORY / sessions
- 管理台改文件下轮生效

Sandbox

隔离

✓容器内 Shell

- Docker · K8s · E2B · Daytona · AgentRun
- Workspace Projection（SHA-256 增量）
- 快照恢复 pip / npm install 状态
- 生产跑不可信代码首选

阿里云

内置上下文压缩策略 - 四道防线

让对话保持在 token 预算内，同时不丢关键信息

压缩流水线（按触发时机排布）

Tool 执行	→	Result Eviction	→	Truncate Args	→	Summary LLM	→	Overflow Recover
产生工具结果		单条 > 80K 落盘 上下文留首尾 +指针		大参数字符串截断 零 LLM 成本		前缀 → 结构化摘要 尾部保留原文		真的撞墙 → 极端压缩 自动重试一次

可调节的杠杆

触发阈值	`triggerMessages · triggerTokens`
保留窗口	`keepMessages · keepTokens`
Flush 时机	`ALWAYS · NEVER · THROTTLED`
独立小模型	`compaction.model / memory.model`
卸载排除	`read_file · grep_files 默认排除`

压缩不会触碰的内容

Plan Mode 状态	`AgentState.planModeContext`
子 agent 后台任务	`agents/<id>/tasks/<sid>.json`
todo_write 清单	`AgentState.tasksContext`
权限规则	`AgentState.permissionContext`
永不压缩对话日志	`sessions/<sid>.log.jsonl`

双层长期记忆- 事实自动沉淀

让 agent 记住跨会话的事实，同时避免上下文无限膨胀

对话 messages
当前上下文

Flush LLM
每次 call 结束

第一层 · 日流水账
memory/YYYY-MM-DD.md · 追加 · 未去重

Consolidation LLM
后台节流 · 最少 30 min/次

第二层 · MEMORY.md
合并去重 · 策划后的长期记忆

每轮注入

System Prompt
<memory_context>

agent 主动查询工具

memory_search | memory_get | session_search

见到 MEMORY.md 截断提示时会自动调用

管线里的三次独立 LLM 调用（定制入口）

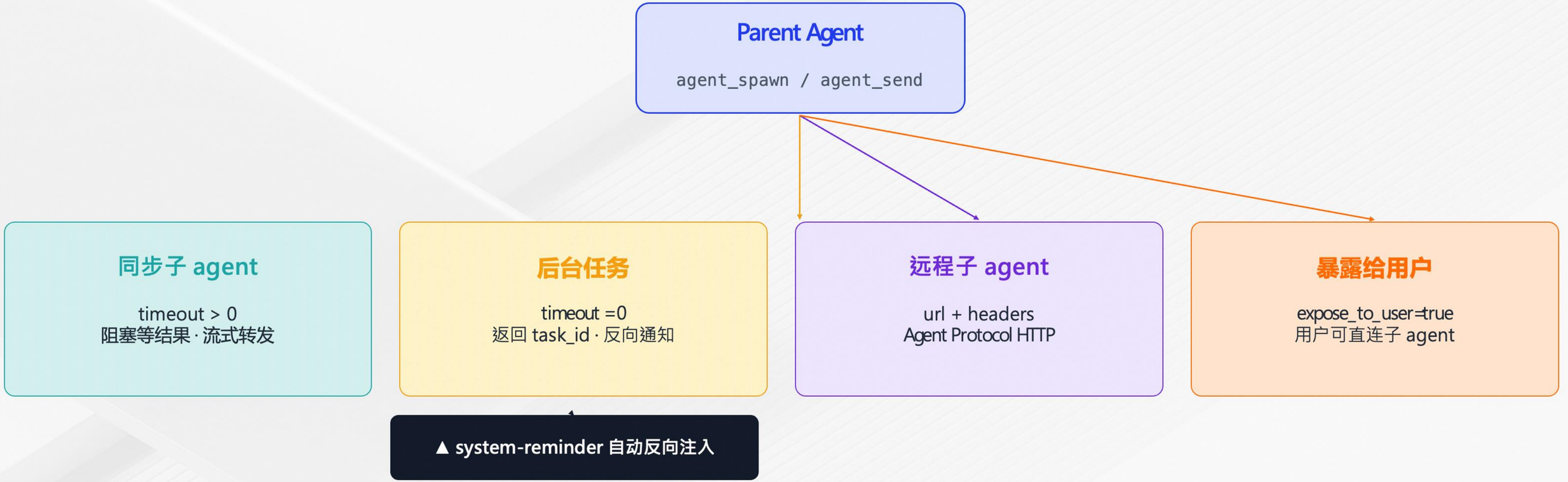
Flush
`MemoryConfig.flushPrompt`
从对话窗口抽取长期事实到日流水账

Consolidation
`MemoryConfig.consolidationPrompt`
把每日流水账合并去重到 MEMORY.md

Compaction Summary
`CompactionConfig.summaryPrompt`
把对话前缀压成一条摘要（当下上下文）

子智能体编排、委派、并行、异步通知

把独立、上下文重、可并行的任务交给专家型子 agent



关键能力

- ISOLATED / SHARED workspace
- 流式事件带 source 路径
- persistSession 复用实例
- 递归 3 层硬上限
- DENY 权限自动继承
- Plan Mode 自动只读

阿里云

沙箱管理：隔离、恢复与分布式

把危险操作关进容器，把安装状态存进快照

01 执行边界

文件与 shell 命令都在隔离容器里执行，宿主完全不参与

02 跨调用恢复

pip install · npm install · 临时文件都随快照保留，下次 call 无需重装重装

03 多副本可用

通过分布式 store +远端快照，任意节点都能 resume 出同一份工工作区

快照后端

- Local · 本地磁盘
- Redis · 低延迟
- OSS/S3 · 多副本
- JDBC · 关系库

call() 生命周期 → 沙箱决策



有 → 复用

最快路径

无容器 有快照 → 恢复

快照重建

都没有 → 冷启动

WorkspaceSpec 全量初始化

后端矩阵 · Docker · Kubernetes · Daytona · E2B · AgentRun

IsolationScope 决定沙箱 slot 复用（USER 默认 · SESSION 天然并发安全 · AGENT/GLOBAL 需 executionGuard 串行化）

Skills：四层注册中心 & 沙箱内执行

SKILL.md + references/ + scripts/ = 可复用的能力包

注册中心 · 四层优先级（低 → 高）

L4 — <userId>/skills/

用户级隔离目录 · 覆盖共用版

L3 — workspace/skills/

工作区共用 · 项目特有约定

L2 — skillRepository(...)

市场后端 · Git / Nacos / MySQL / Classpath

L1 — projectGlobalSkillsDir

项目全局目录 · ~/.agentscope/skills/

▲ 高

▼ 低

沙箱内执行 · 三步透明化

01 物化 Stager

市场 skill 从内存写到宿主 .skills-cache/
· SHA-256 文件级去重
· 启发式恢复 chmod +x

02 Workspace Projection

把 skills/ + .skills-cache/ 打成 tar
· hydrate 进沙箱 /workspace
· 内容 SHA-256 整体比对增量

03 容器内执行

<files-root> 用容器内绝对路径
· agent 不用猜路径
· 快照保留 pip/npm install 副作用

阿里云

计划模式：想清楚 -> 写下来 -> 再动手

只读思考阶段 +计划文件 +HITL 退出

工作流（明确固化四步：设计 → 写计划 → 人确认 → 执行）



Plan 阶段的白名单

只读文件工具	`read_file` · `grep_files` · `glob_files` · `list_files`
内存查询工具	`memory_search` · `memory_get` · `session_search`
Plan 三件套	`plan_enter` · `plan_write` · `plan_exit`
任务清单	`todo_write`
Shell（可选）	`.allowShellInPlanMode()` 后放开为只读用途

四种终态（别只看 isPlanModeActive）

未进入	模型直接 build，任务与 workspace 不匹配
进入 → plan_exit	成功：规划完 +获批 → build 模式
仍 plan · 有 PLAN	已起草但未退出；后续消息可批准继续
仍 plan · 无 PLAN	只说不做：文本像计划但没写出

Channel：消息平台 -> Gateway -> Agent

一行接线，自动完成会话管理 · 并发 · 路由 · 流式

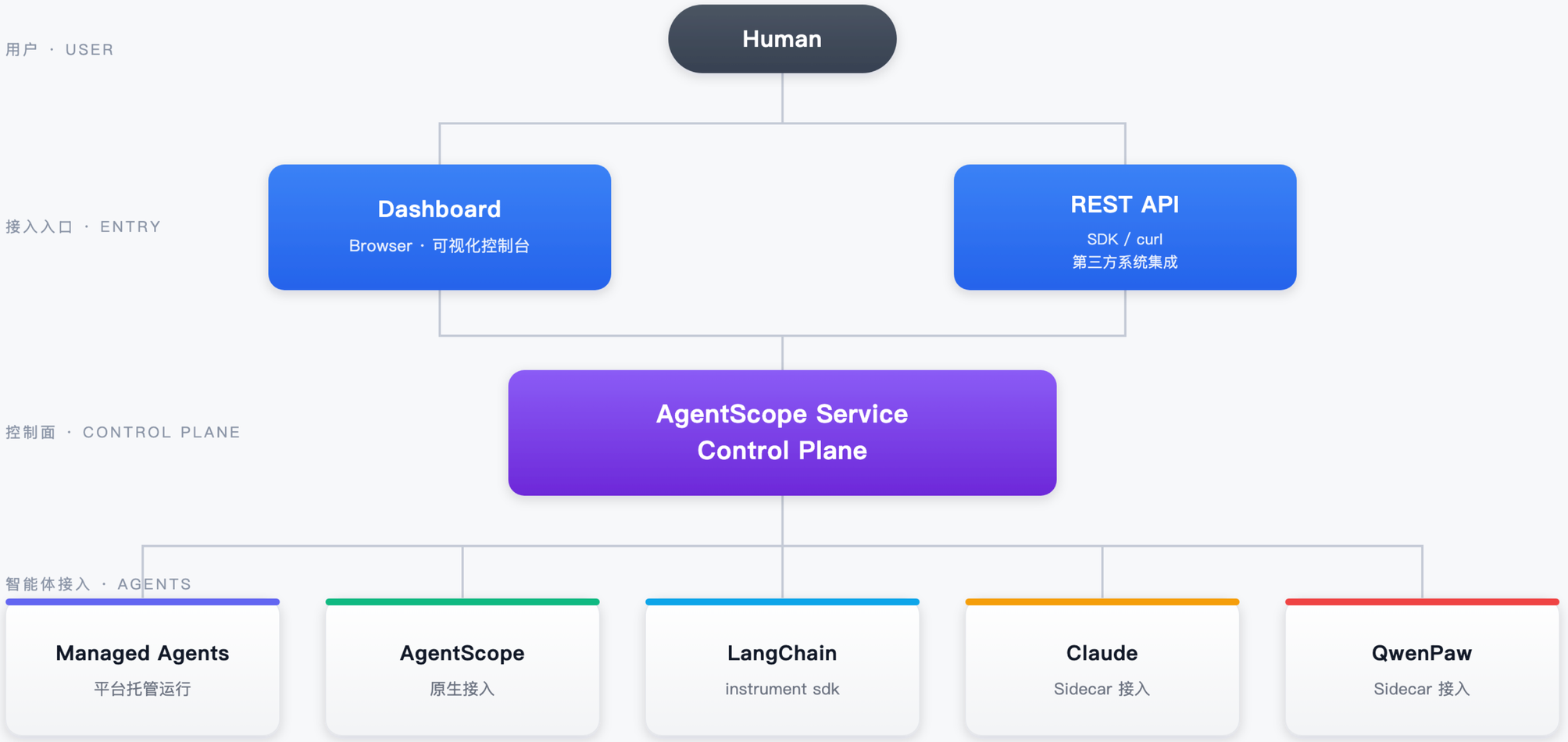


阿里云

3 AgentScope Service 企业级智能体控制面

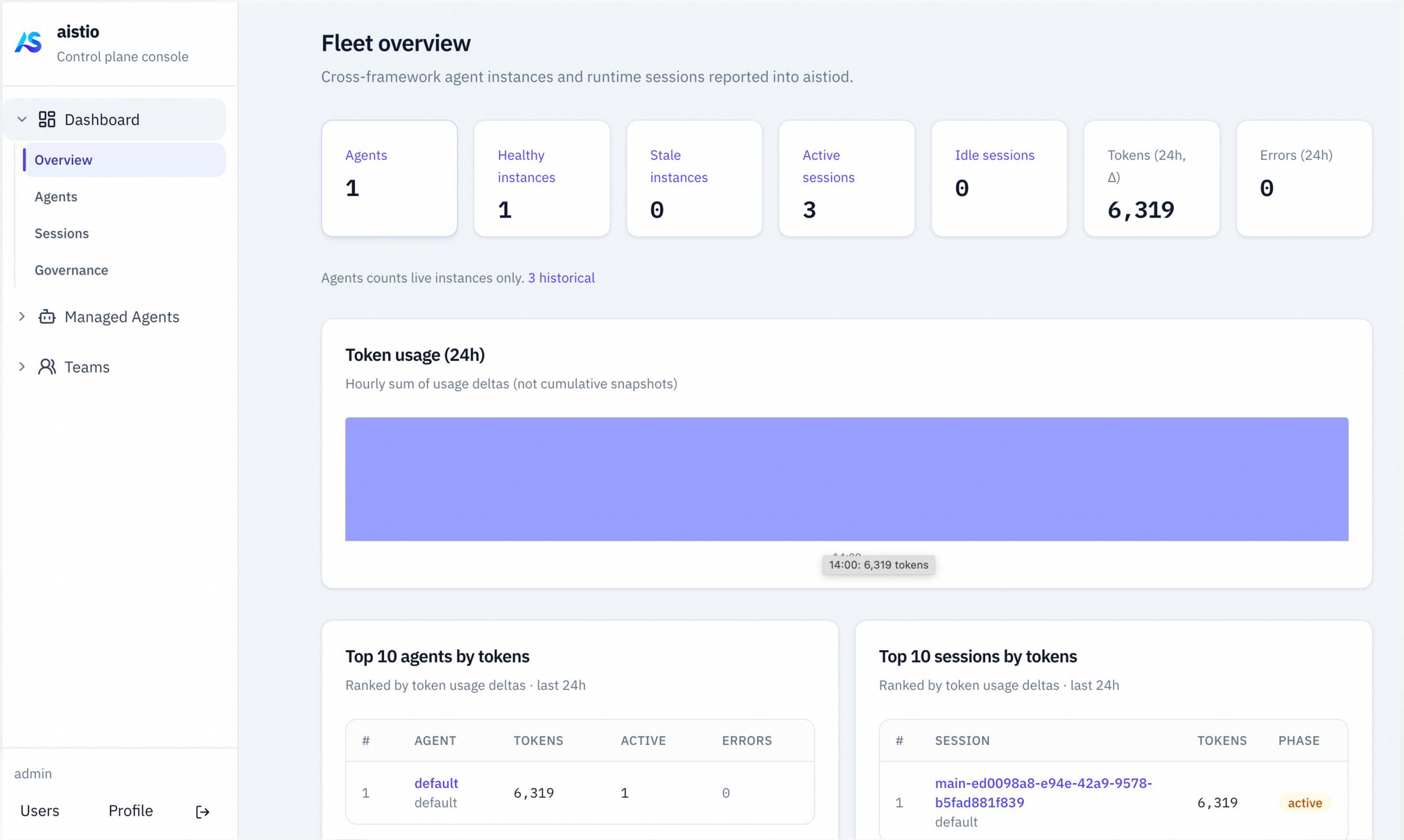
AgentScope Service -企业级智能体编排控制面

AgentScope Service 的目标不是替换你现有的 Agent 框架，而是提供一层统一控制面，让你把用不同框架（AgentScope、Langchain、ADK 等）、不同技术栈（Claude、OpenClaw、QwenPaw 等）搭建的智能体统一管控、协作起来。



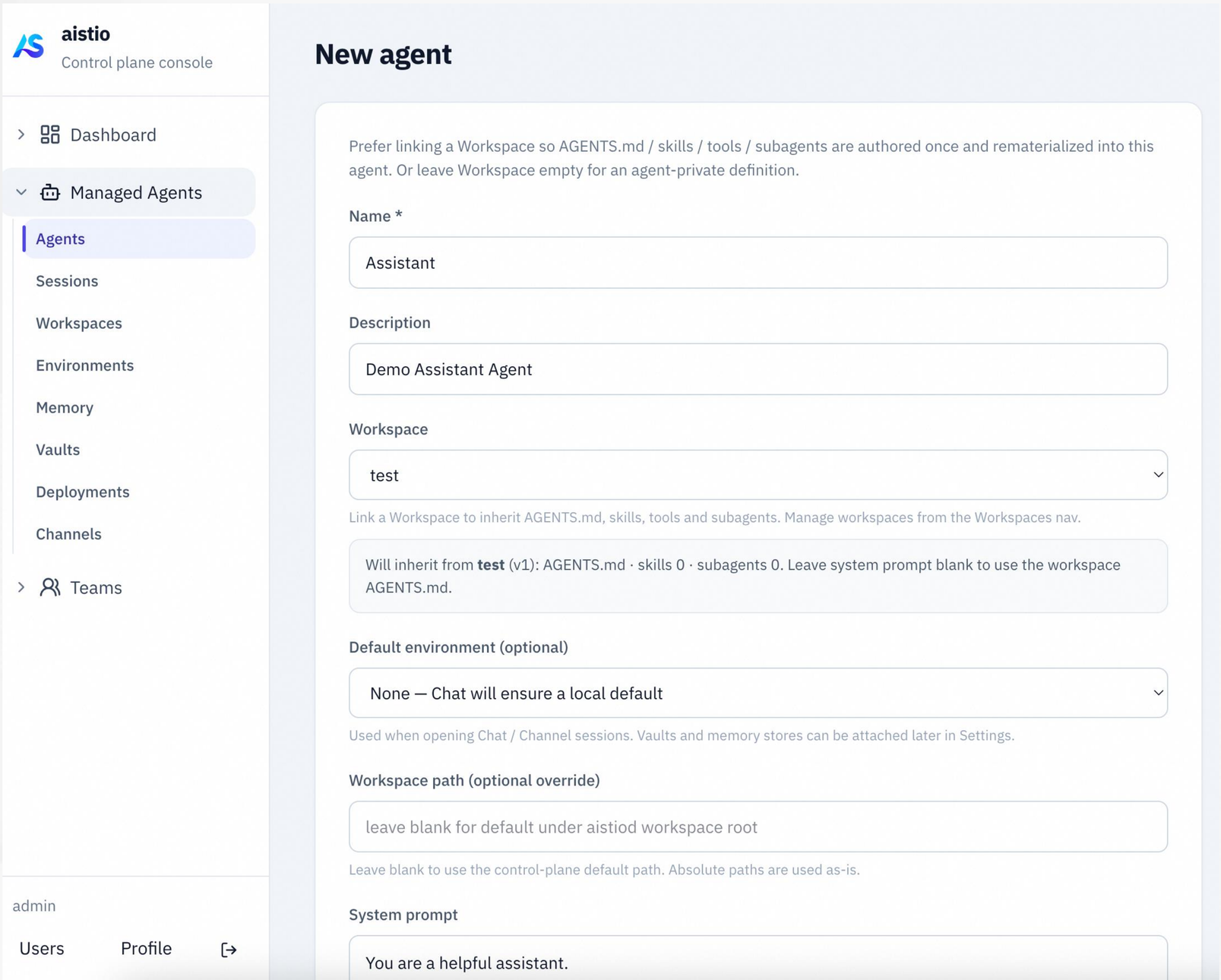
AgentScope Service · 系统架构图

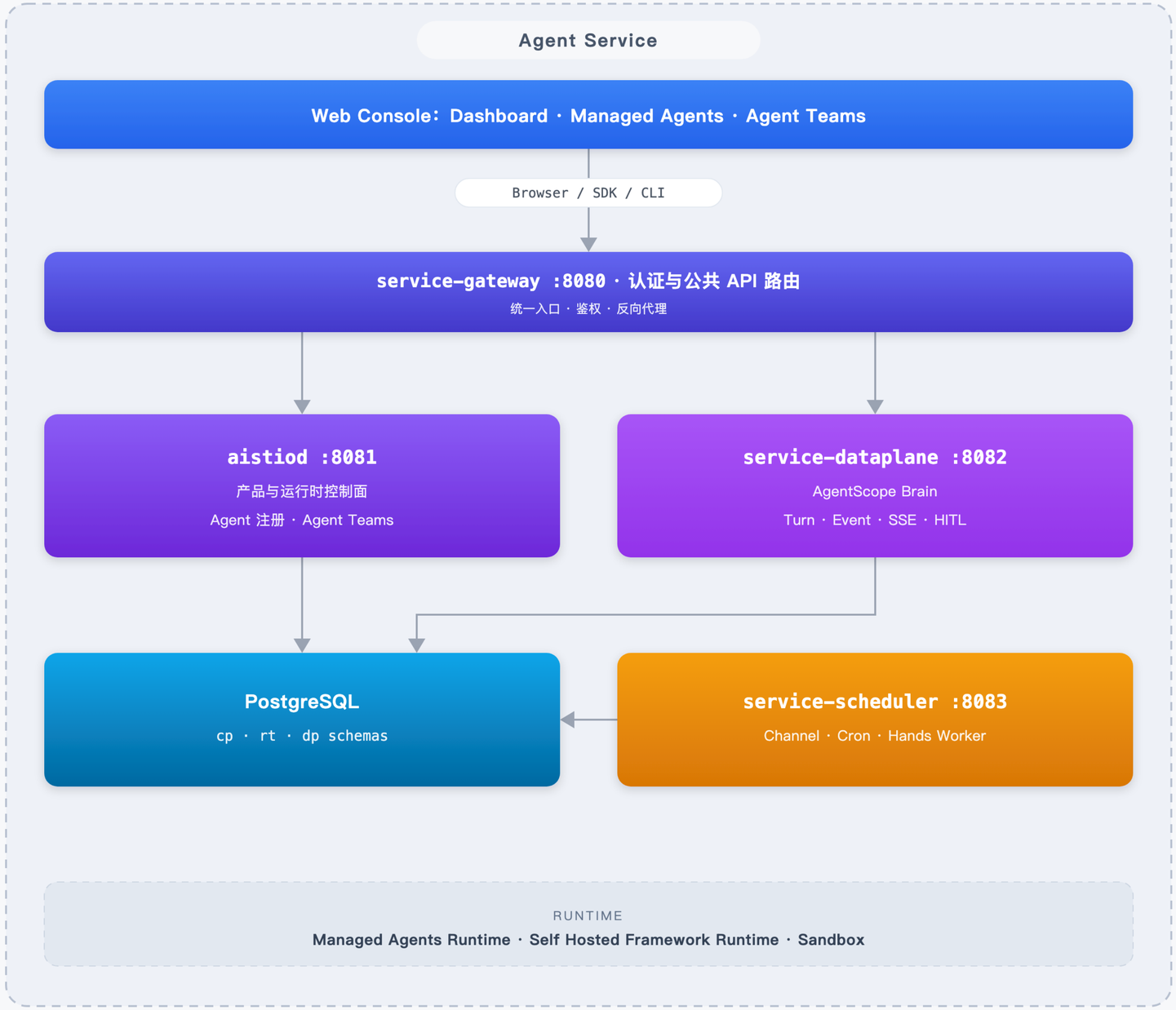
控制面 & Dashboard



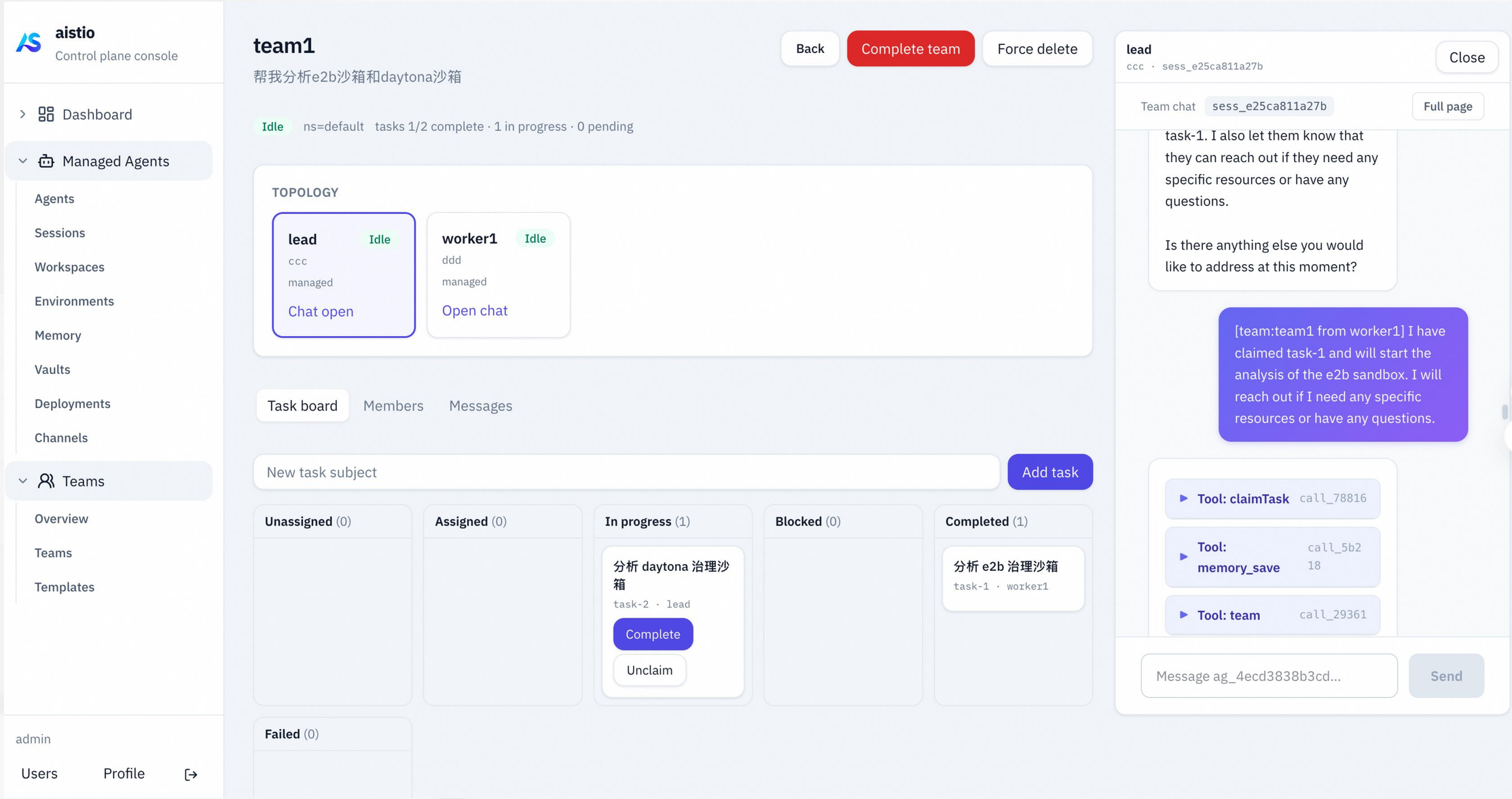
阿里云

Managed Agents 实现 Harness 全托管




Agent Service
Web Console：Dashboard · Managed Agents · Agent Teams
Browser / SDK / CLI
service-gateway :8080 · 认证与公共 API 路由
统一入口 · 鉴权 · 反向代理
aistiod :8081
产品与运行时控制面
Agent 注册 · Agent Teams
service-dataplane :8082
AgentScope Brain
Turn · Event · SSE · HITL
PostgreSQL
cp · rt · dp schemas
service-scheduler :8083
Channel · Cron · Hands Worker
RUNTIME
Managed Agents Runtime · Self Hosted Framework Runtime · Sandbox

实现多 Agent 协作


aistio
Control plane console
Dashboard
Managed Agents
Agents
Sessions
Workspaces
Environments
Memory
Vaults
Deployments
Channels
Teams
Overview
Teams
Templates
admin
Users
Profile
team1
帮我分析e2b沙箱和daytona沙箱
Back
Complete team
Force delete
Idle
ns=default
tasks 1/2 complete · 1 in progress · 0 pending
TOPOLOGY
lead
Idle
ccc
managed
Chat open
worker1
Idle
ddd
managed
Open chat
Task board
Members
Messages
New task subject
Add task
Unassigned (0)
Assigned (0)
In progress (1)
分析 daytona 治理沙箱
task-2 · lead
Complete
Unclaim
Blocked (0)
Completed (1)
分析 e2b 治理沙箱
task-1 · worker1
Failed (0)
lead
ccc · sess_e25ca811a27b
Close
Team chat
sess_e25ca811a27b
Full page
task-1. I also let them know that they can reach out if they need any specific resources or have any questions.
Is there anything else you would like to address at this moment?
[team:team1 from worker1] I have claimed task-1 and will start the analysis of the e2b sandbox. I will reach out if I need any specific resources or have any questions.
Tool: claimTask
call_78816
Tool: memory_save
call_5b218
Tool: team
call_29361
Message ag_4ecd3838b3cd...
Send

扫码加入钉钉群

阿里云 | 计算，为了无法计算的价值